

Donghyung Lee

DevOps Engineer | Career Description

Core Competencies

Cloud Infrastructure Design & Operations

Multi/hybrid cloud architecture design and operations experience on AWS and GCP

- AWS EKS, GCP GKE production operations & cost optimization (20% reduction via hybrid transition, 50% via GCP migration, savings reinvested in new projects)
- Terraform IaC infrastructure automation & version control (100% GCP resource codification)

CI/CD Pipeline & GitOps

Maximizing development productivity through GitOps-based deployment automation

- CI/CD pipeline design & enhancement with GitHub Actions, GitLab CI, ArgoCD
- 50-67% deployment time reduction with continuous GitOps deployment shortening Time to Market, 95% rollback time decrease (30min → 1-2min)

Monitoring & Observability System Implementation

Establishing proactive incident detection and rapid response systems

- Evolved from ELK → EFK Stack, then redesigned into ECK Operator-based Elastic Stack 9.0 CRD declarative management (automatic TLS rotation, automated rolling upgrades)
- kube-prometheus-stack with custom alert rules, Grafana dashboards, and 4 DB exporters (MySQL, Redis, Elasticsearch, PostgreSQL) integrated

Automation & Efficiency

Operations automation and developer productivity tools using Python and Bash

- Built an internal LLM service (Ollama + Open WebUI) — 100% internal security guideline compliance, improved dev team productivity
- Established self-sufficient Kubernetes ops at DP (Delivery Partner) sites — designed and ran a 6-month training program (avg 10 attendees, 4h/session), standardized materials reused across other clients

Kubernetes Platform Modernization & Open-Source Contribution

Zero-downtime migration of network/logging stacks and releasing reusable public charts as open source

- Migrated multiple ingress-nginx instances to a single NGINX Gateway Fabric control-plane + per-class Gateway CRs with zero downtime (full cutover, wildcard TLS consolidated)
- Released 3 open-source Helm charts (nginx-gateway-cr, elasticsearch-eck, kibana-eck) and multiple composite GitHub Actions on ArtifactHub / GitHub Marketplace

Data Engineering & Internal LLM Services

End-to-end ownership from GCP data pipelines to in-house LLM services

- Built a multi-source (MongoDB · CloudSQL · GA · Dune) → BigQuery → Google Sheets pipeline by combining BigQuery, Dataflow, Cloud Functions, and Cloud Scheduler — entire infrastructure 100% Terraform IaC
- Deployed an internal LLM service (Ollama + Open WebUI) on a Kubernetes-hosted on-premises GPU server, saving ~\$100/month by replacing external AI SaaS subscriptions and ensuring 100% internal security compliance

Professional Experience

Phantom (Concrit Studio)

2024.10 ~ Present

DevOps Engineer

As the sole DevOps infrastructure engineer at a game development company, I design, build, and operate AWS-based cloud and on-premises servers. Development and test servers are on-premises, while QA and Production servers are on AWS, operating a hybrid architecture that reduced monthly infrastructure costs by 20% (\$60K → \$48K) and channeled the savings into new project investments.

Currently providing secondary technical support and operations for a live-service game in collaboration with the external publisher, and building on-premises/AWS infrastructure to improve development productivity for new game projects.

Project 1: AWS-On-premises Hybrid Cloud Architecture

Contribution 100% (Solo design & implementation, in collaboration with the external publisher's ops team) 2024.10 ~ Present (In Operation)

Problem

While operating the entire infrastructure on AWS, high cloud costs of \$60,000/month necessitated cost optimization.

Approach

Workload-specific cost analysis revealed that dev/test environments had minimal traffic variation, making cloud auto-scaling unnecessary, while only QA/Production required scalability and stability. Designed a hybrid architecture transitioning dev/test to on-premises while keeping QA/Production on AWS.

The two environments are operationally independent without VPN (application traffic isolated); the image registry is also split by environment (dev = Harbor, QA/Production = ECR), with each environment building independently to match its architecture. Unified operational standards through Terraform and Ansible IaC integration (100% IaC coverage for new components, IAM excluded) across both environments, achieving consistent infrastructure management without increasing operational complexity.

Troubleshooting

- When configuring ALB Ingress on EKS, needed to route to different backends per game client version.
Resolved by combining custom header-based routing rules with Target Groups in ALB conditional routing
- During rolling deploys of the ArgoCD-managed production services, the ALB target deregistration delay didn't line up with Pod SIGTERM handling, dropping some session connections.
Achieved zero-downtime deploys with 0 connection drops via 3 changes — (1) aligned ALB drain timing: a preStop hook + readiness-first fail block new requests during termination while in-flight requests are served to completion, (2) tuned the RollingUpdate strategy surge-first to fit replica scale, holding capacity through rollout, (3) applied a PDB to multi-replica services so Karpenter consolidation never evicts Pods simultaneously

Results

- Workload analysis-based infra optimization reducing monthly costs by 20% (\$60,000 → \$48,000), reinvesting ~\$144,000 annual savings into new project infrastructure for resource reinvestment cycle
- Offloaded dev servers to on-premises, cutting ~\$1,000/month in AWS resource costs and optimizing resource usage
- Ensured operational stability and fault isolation through environment role separation

Project 2: On-premises CI/CD Pipeline Construction & Enhancement

Contribution 100% (Full infra & pipeline ownership) 2024.12 ~ Present (In Operation)

Problem

Developers manually built and deployed to servers, taking an average of 30 minutes per deployment with frequent failures due to human errors.

Approach

Built a GitOps-based automated deployment environment combining GitLab CI and ArgoCD. Developers simply Git Push to trigger automatic build→test→deploy, with ArgoCD monitoring Kubernetes cluster state for declarative deployments.

Built Docker images with Kaniko without Docker-in-Docker (no privileged container), and configured multi-environment (alpha/review/dev/staging) deployments in a single pipeline.

Troubleshooting

- Applying `selfHeal=true` uniformly meant prod changes auto-applied without human approval and left little room for emergency response → split by environment: prod uses `selfHeal=false` + manual sync windows to gate deploys while dev/staging keep `selfHeal=true` for automatic drift correction, with every sync event wired to Slack for both change traceability and stability
- Moved the data-upload path to keyless — replaced the direct SSH scp/rsync with staging to an S3 transit bucket, then having the target instance run `aws s3 sync --exact-timestamps` via SSM SendCommand, eliminating SSH keys entirely and removing deploy-account key-leak risk

Results

- 67% deployment time reduction (30min → 10min), and cut the data-deployment source fetch from ~139s to ~30s by replacing it with `git fetch --depth=1 --filter=blob:none`
- Split deployment via GitOps automation — dev/qa/review run continuous merge-driven deployment (~100/week per core service repo) to cut new-feature Time to Market, while prod ships controlled ~5-7 releases/week gated behind manual sync windows for stability
- Replaced the manual build/deploy/config-apply steps with the pipeline to eliminate human-error deployment failures, and the cross-repo MasterData pipeline auto-fans-out to downstream services on data changes while preventing circular (loop) triggering

Project 3: Central Logging & Observability Platform (Collection → Integrity/HA → Product Analytics)

Contribution 100% (Sole design across the full lifecycle — collection · integrity · product analytics) 2024.11 ~ Present

Starting from a log-scattered on-premises environment, advanced centralized logging through three stages (ELK → EFK → ECK Operator), hardened collector data integrity / HA, then repurposed the accumulated EFK log pipeline into a game user-retention analytics platform. Solely designed and operated the full observability lifecycle — build → operate → improve → analytics.

3-1. Centralized Logging System (ELK → EFK → ECK Operator 3-stage Enhancement, 2024.11 ~ Present)

Problem

Server and container logs were distributed across the on-premises environment, making root cause analysis time-consuming during incidents, with no unified monitoring system in place.

Approach

Enhanced the logging stack in **three stages: ELK → EFK → ECK Operator**. Built centralized logging on ELK first, then migrated to an EFK stack (lightweight Fluent Bit DaemonSet collection + Fluentd processing) to resolve Logstash's JVM memory burden and scaling limits.

As Elastic's official Helm chart stalled for over two years, moved to CRD-based declarative management (ECK Operator), performing a zero-downtime major upgrade of Elastic Stack from 8.5.1 to 9.0.0, and templated the on-premises EFK stack into AWS(EKS) `-aws` stacks to port the same model — automating log lifecycle (ILM) and S3 snapshot retention with keyless IRSA.

Results

- 90% log search time reduction (per-Pod access → single Kibana interface), improved MTTR keeping service availability stable
 - Redesigned from heavyweight Logstash to a lightweight Fluent Bit + Fluentd architecture, and the ECK Operator migration removed manual operations for TLS certificates, accounts, and upgrades (rolling upgrades via a single-line CR spec.version change), completing a stalled 2-year Elastic Stack major upgrade with zero downtime (8.5.1 → 9.0.0)
 - Templated the on-premises EFK stack into five AWS(EKS) `-aws` stacks, porting the same logging model as-is — automating index lifecycle (ILM) and S3 snapshot retention with keyless IRSA web-identity
-

3-2. Fluent Bit Log Collector Data Integrity & HA Hardening (2026.03 ~ 2026.05)

Solved log re-index/loss risk from Fluent Bit tail-offset loss on Pod restart with a SQLite offset DB and dedicated PVC — zero re-index / zero duplicates, 7 alerts pre-empting silent collector failures

3-3. Game User Retention Analytics Platform (EFK Logs → Product Analytics, 2026.06 ~ Present)

Extended a search-only EFK stack into a product-analytics platform via an Elasticsearch Transform (5-min) pivoting events to one row per user — a 12-panel Kibana dashboard delivering D+1~D+30 retention and cohorts, no separate SaaS

Project 4: Developer Productivity Tools (APK Deploy Bot · Doc Automation · LLM Service · Git Mirroring · Static File Server)

Contribution 100% (Full system design & development) 2025.02 ~ Present

Designed, developed, and operated 5 internal tools to automate repetitive manual tasks and boost development team productivity.

4-1. Slack APK Distribution Automation Bot (4 weeks, 2025.02)

Deployed a Python bot on Kubernetes auto-sending completion messages and download QR codes to Slack on Jenkins APK build finish — 90% QA delivery-time reduction, version confusion eliminated

4-2. GitLab-Google Drive Document Automation System (2 weeks, 2025.06)

Integrated GitLab Webhook with Google Drive API to auto-convert markdown into Google Docs on push, optimized with rclone `--checksum` — 80% doc-management reduction (10min→2min), GitLab unified as SSOT

4-3. Internal LLM Service Deployment (4 weeks, 2025.11)

Deployed Ollama and Open WebUI on Kubernetes on an on-prem GPU server with NFS model storage — 15 developers running ~100 queries/day, saving ~\$100/month over external SaaS and removing data-leak risk

4-4. Git Repository Mirroring Tool + Retry API Follow-up (4-week initial build, 2026.02 ~ 2026.06)

Problem

Manual sync across CodeCommit / GitLab / GitHub caused omissions and delays, so internal source needed consolidation into GitLab — and mirror failures were recoverable only by `kubect1 exec`, which exposed the webhook secret.

Approach

Built git-bridge, a Go-based bidirectional Git mirroring tool, running event-driven incremental sync with loop detection and deployed on Kubernetes.

Later authenticated a manual-retry `POST /retry/mirror` endpoint with a dedicated token `RETRY_API_TOKEN` (constant-time compared) separated from the webhook secret, enabling recovery with no secret exposure, and blocked non-fast-forward regressions with `ref_overrides` pinning sync direction per ref pattern plus a **fail-open principle**.

Results

- Fully automated sync of ~30 events/day across 10 repositories, eliminating manual mirroring; open-sourced
- The Retry API gave an SRE-friendly recovery path with no webhook-secret exposure, and the `ref_overrides` direction-pinning guard blocked non-fast-forward regressions — 6/6 PASS on a test-only repo, production isolated

4-5. Static File Server - In-house Development (Open Source, 2026.03 ~ Present)

Problem

The previously used [halverneus/static-file-server](#) offered only basic file serving, lacking the operational features QA APK distribution and internal file sharing needed, and its upstream chart maintenance had effectively stalled.

Approach

Designed and developed a lightweight Go-based static file server (static-file-server) in-house, implementing the operational features it needed, and operated a [Helm repository on GitHub Pages](#) to establish a deployment standard.

Deployed to Kubernetes with an in-house Helm chart connected to NFS storage for stable long-term artifact hosting, completing a full replacement of the legacy halverneus/static-file-server deployment.

Results

- Released as open source under Apache-2.0, handling ~30 downloads/day and 5 APK builds/week
- Converged APK distribution, document sharing, and build-artifact delivery into a single tool, and removed the upstream dependency so security patches and new features run on an in-house cycle

Project 5: Kubernetes Cluster Operations Enhancement

Contribution 100% (Solo design — monitoring · upgrade automation · Helm management framework · NGF migration · 3 OSS Helm charts · storage performance optimization · Shell→Python migration; captured as 4 automation scripts establishing a standard procedure → so the team can reuse) 2025.12 ~ Present

To see into the cluster better and keep it running reliably, overhauled monitoring, automated K8s upgrades and Helm management, and migrated the network controller onto the Gateway API.

Released the artifacts from the NGF migration and ECK Operator adoption as OSS Helm charts, resolved storage bottlenecks via control-plane / DB-node SSD migration and build I/O isolation, and modernized the infra deploy/upgrade pipeline with wave-based automation + a Shell → Python migration.

5-1. Monitoring & Alerting Enhancement (2025.12 ~ Present)

Designed custom alerts and Grafana dashboards on kube-prometheus-stack, integrating DB, node-exporter, and Cilium metrics — 27 custom alert rules across 9 component groups, severity routing and inhibit rules removing duplicates

5-2. K8s Upgrade Automation & Helm Chart Management Framework (2025.12 ~ Present)

Built a Kubespray upgrade automation script and a multi-step health check (node/etcd/API Server), standardizing version management with a Helm framework and 4-canonical-template synchronizer — 90% verification-time reduction, automated cert renewal removing expiration outages

5-3. Ingress-nginx → NGINX Gateway Fabric (NGF) Migration (2026.04)

Migrated the on-prem ingress-nginx fleet to a single NGF 2.x control-plane + per-class Gateway CRs — MetalLB IPs preserved so DNS/firewall unchanged, incident-free cutover (30-60s per class), one wildcard cert cutting manual renewal 50%

5-4. Three Open-Source Helm Charts — nginx-gateway-cr · elasticsearch-eck · kibana-eck (2026.04 ~ 2026.05)

Refined NGF and ECK outputs into three reusable charts (nginx-gateway-cr-elasticsearch-eck-kibana-eck), published on ArtifactHub under Apache-2.0 — cut the mgmt cluster over to public OCI charts with cluster_uuid unchanged and zero downtime

5-5. Storage Performance Optimization — etcd / DB SSD Migration + Build I/O Isolation (2 weeks, 2026.04 ~ 2026.05)

Problem

The control-plane VM and the game-DB worker VM both ran on HDDs, contending with GitLab Runner buildx jobs for disk I/O, so etcd WAL fsync p99 spiked to ~2s, apiserver GET p99 to ~7s, and game DB COMMIT latency to 9~14s — causing recurring play-blocking incidents. Root cause analysis pinned it on GitLab Runner's heavy write traffic saturating the game-DB worker VM's disk fsync queue.

Approach

Tuned MySQL fsync parameters to immediately ease the disk fsync load, then migrated the control-plane VM to SSD with **8.5 minutes of downtime (vs. a 15-minute estimate)**, verifying via etcd health checks and Prometheus metrics. Losslessly migrated the game-DB worker VM's MySQL and Redis via backup → SSD migration → restore.

Finally, stood up a dedicated build-only VM on a separate physical server and isolated GitLab Runner buildx jobs onto it via taint/toleration to structurally prevent I/O contention, packaging the work into automation scripts so future worker nodes follow the same procedure.

Results

- **Diagnosed GitLab Runner buildx disk-I/O contention as the root cause** and migrated the control-plane VM to SSD, bringing etcd fsync and apiserver latencies — which had spiked into the seconds range — back to normal, restoring Kubernetes API availability
- **Game DB COMMIT latency dropped from 9~14s back to the ms range**, taking play-blocking incidents during builds down to zero, and a dedicated build-only VM physically isolates buildx workloads from DB workloads, structurally preventing I/O contention recurrence
- Captured the work as 4 automation scripts + config files so future worker-node additions reuse the same standard procedure (ops standardization)

5-6. Infra Deploy/Upgrade Automation + Shell → Python Migration (2026.03 ~ Present)

Problem

The internal Kubernetes infra monorepo's 25 components were managed by ~39,000 lines of helmfile + shell, leaving new-component deploys, chart upgrades, and template sync entirely manual; shell + awk hit walls on multi-environment branching, complex YAML patching, and testability, and CI took ~5 minutes just to install helmfile and its plugins.

Approach

Structured the CI pipeline into 6 waves (bootstrap → core → security → db-redis → observability → apps), detecting MR diffs to helmfile diff/apply only changed components, with a manual gate standardizing prod-safe sequential deploys. Automated upgrades too, so CI detects new versions across the 25 component charts and opens an auto-MR rendering changed chart, values diff, and breaking-change notes. Migrated the existing shell + awk automation to Python (stdlib only), modularizing canonical templates, sync/backup scripts, and CI scripts with per-component upgrade modules and 758 unittests, and applied Project 7's helmfile-tools image to bring CI bootstrap time from 5min to ~10s.

Results

- **Wave-based automated deploy of 25 components + auto-upgrade MR generation**, slashing the operational burden of infra deploys and upgrades
- **Migrated the shell + awk automation to Python (stdlib only)** — removing the walls of multi-env branching and complex YAML patching while gaining readability and testability (758 unittests passing)
- Adopting the helmfile-tools image brought **CI bootstrap time from 5min to ~10s (~95% reduction)**, and local (macOS venv) + remote (CI container) execution share the exact same code, removing environment-drift issues between dev and deploy

5-7. helmfile Push → ArgoCD Pull-GitOps Migration (App-of-apps Control Plane, 2026.06 ~ Present)

Problem

Platform releases were deployed via helmfile push, tying the deploy actor to the CI runner.

As the estate expanded to multiple clusters, a pull-based GitOps control plane was needed to declaratively converge per-cluster drift.

Approach

Migrated helmfile-push deployment to ArgoCD pull-GitOps. A Matrix + Git-files generator ApplicationSet (a controller that auto-emits Applications) reads each component's `argocd/*.yaml` marker files to create Applications, registering 24 platform releases (16 on-premises + 8 AWS) across 2 clusters.

Existing helm releases were handed over to ArgoCD without re-creation (zero-downtime adoption) with deploy order preserved, and App-of-apps prune-cascade guardrails plus least-privilege AppProjects shrink accidental mass-deletion risk and blast radius.

Results

- Migrated from helmfile push to ArgoCD pull-GitOps, managing 24 platform releases across 2 clusters (16 on-premises + 8 AWS) from a single declarative control plane, adopting existing helm releases without re-creation (zero-downtime) and preserving deploy order
- App-of-apps prune-cascade guardrails eliminate the risk of mass GitOps deletion, and least-privilege AppProject whitelists shrink blast radius

Project 6: Infrastructure Security System

Contribution 100% (Design, deployment, SSO integration) 2026.04 ~ Present

Building security infrastructure for safe management of sensitive information required for infrastructure operations.

6-1. Vaultwarden Password Management System (1 week, 2026.04 ~ Present)

Centralized internal secrets scattered across Slack/email/personal vaults with Vaultwarden — deployed on Kubernetes via Helm with GitLab SSO (OIDC) and RBAC, migrating 70 users and 50 secrets at zero SaaS cost

6-2. Keycloak Operator-based Central SSO Introduction (2026.04 ~ Present)

Problem

Internal infra tools — ArgoCD, Harbor, Vaultwarden — were each using GitLab directly as their OIDC IdP, layering more and more dependence on GitLab's own user / session / MFA policy.

To prepare for central management of LDAP / MFA / session policies, a dedicated central auth broker (Keycloak) had become necessary.

Approach

Introduced the Keycloak Operator + KeycloakCR + KeycloakRealmImport with a PostgreSQL backend, then configured Keycloak to broker GitLab login so existing accounts kept logging in. Codified Realm / Client / IdP / Group / Mapper setup via a bootstrap script and migrated Harbor, ArgoCD, and Vaultwarden's OIDC from direct-GitLab to via-Keycloak.

For RBAC, GitLab groups are brokered through Keycloak group mappers and wired into ArgoCD role mappings, consolidating the source of truth for permissions onto Keycloak.

Results

- **ArgoCD, Harbor, and Vaultwarden OIDC integrations completed with zero downtime and verification checks PASS** — removed direct dependence on GitLab
 - Established a single authorization flow from GitLab groups (`server` , `global-admin`) → Keycloak group mappers → ArgoCD roles, consolidating authorization SSOT onto Keycloak, and laid the groundwork for future LDAP federation, MFA enforcement, and central session policy
 - Captured five pitfalls hit during Keycloak Operator adoption (IdP providerId / scope / mapper config / Operator idempotency / Harbor user mapping) into the plan doc to prevent recurrence on future cluster rebuilds
-

6-3. GitOps Secret Management — External Secrets Operator · Sealed Secrets (2026.05 ~ Present)

Replaced plaintext DB passwords in Git by wiring External Secrets Operator to AWS Secrets Manager for ExternalSecret references and standardizing the Harbor pull secret as a cluster-wide SealedSecret — making AWS SM the single source of truth

6-4. OpenVPN Remote-Access Server (2026.07)

Built an OpenVPN remote-access server in-house on a physical server to lift the iptime router's built-in VPN 5-concurrent-connection limit — easy-rsa PKI with per-client EC certificate (prime256v1) auth + tls-crypt encrypting and authenticating the control channel, AES-256-GCM / TLS 1.2+, full-tunnel by default with split-tunnel switchable, and the 10.8.0.0/24 tunnel range routed to the internal physical-server network via SNAT (MASQUERADE) (configured for up to 50 concurrent connections).

Automated operations with an idempotent bash install script (PKI build → server.conf render → NAT systemd unit → service start) and a client-certificate issue/revoke/CRL-refresh lifecycle script, controlled remotely from the admin Mac over SSH

Project 7: Shared Toolchain Image Layered Architecture + Auto Version Cycle

Contribution 100% (Architecture design · CI standardization · automation pipeline) 2025.09 ~ 2026.05

Problem

Across the 15 repos in the GitLab CI standardization scope (1 template provider + 14 consumers), each job re-installed helm / helmfile / kubectl / crane / aws-cli on top of an Alpine base, making first-job bootstrap take an average of ~5 minutes. Tool versions were scattered across the 14 consumer repos, so adding a new version effectively required a coordinated bump across every consumer repo.

Reliance on `:latest` tags also undermined reproducibility and made CVE tracking hard.

Approach

Layer 1 (Mirror): copied external registry images (Docker Hub, GHCR) into the internal Harbor via `crane copy` , preserving multi-arch manifests.

Layer 2 (Custom): built 7 fat images on top of Layer 1 (helmfile-tools, node-build, go-build, rclone-backup, aws-cli-tools, kaniko-build, terraform-tools) so consumer repos get the tools immediately on image pin alone.

Tier 1~2 SSOT: introduced a shared `.toolchain_build_custom` template + central `TOOLCHAIN_*_TAG` variables so consumer repos no longer track image tags directly.

Shared CI template SSOT: extracted the CI jobs that had been copy-pasted per repo into a central shared CI template repo, with each repo referencing them via `include`. Manages the auth (keyless AWS auth) / build (kaniko) / deploy (ArgoCD) / backup (Google Drive) job templates plus shared anchors (git-ssh-rules-job-config-packages) from a single source, eliminating copy-paste drift.

Phase 4 auto cycle: a cron job drives the following 4 steps so the full cascade completes with just two manual clicks (pipeline trigger + Tier 2 MR merge):

- (1) Upstream version detection
- (2) Layer 1 mirror auto-MR
- (3) Layer 2 rebuild + Tier 2 `TOOLCHAIN_*_TAG` sync auto-MR
- (4) Auto-propagation to 14 consumer repos (excluding the template provider repo)

Results

- **First-job bootstrap time cut from 5min to ~10s (~95% reduction)**, with 100% tool-version consistency across 14 consumer repos (15 repos total), compressing 12+ scattered version points into a 4-tier governance flow (ARG → central var → consumer indirection → lint drift check)
- The Phase 4 cron auto cycle lands new versions of helm / helmfile / kubectl / crane / rclone (5+ tools) with just 2 manual clicks, and a minor-guard policy (helm/helmfile/kubectl) prevents compatibility breakage in advance
- Built a risk-tiered auto-upgrade pipeline across 25 components (T1 weekly / T2 biweekly / T3 monthly + pre-flight) — version detection → auto-MR → Slack alert → wave-sequenced apply, with a MAJOR_PIN guard and T3 atomic rollback — and built amd64 + arm64 multi-arch images via docker buildx while mirroring external images into Harbor with crane, removing external-registry dependency

Project 8: AWS EKS Production Game-Launch Platform (New Externally-Published Title)

Contribution 100% (Solo greenfield EKS design & build) 2026.06 ~ Present (Building & Operating)

Problem

Separately from the live game's estate (AWS QA·Prod), the new externally-published game ran only as a single on-premises development cluster, and launching it to production required a separate, scalable, highly-available AWS EKS production environment — on-premises alone could not handle global traffic and autoscaling needs.

Approach

Built a greenfield EKS cluster in eu-central-1, expanding the estate into an on-premises dev + AWS production multi-cluster hybrid, and managed ~11 platform components entirely via GitOps (ArgoCD).

Autoscaling / networking: Chose Karpenter over Cluster Autoscaler (unified spot across families, faster node provisioning) to run interruption-resilient multi-family spot autoscaling plus a dedicated on-demand NodePool for game servers, while AWS Load Balancer Controller (ALB/NLB) + ExternalDNS (Route53) + a wildcard ACM cert automate ingress, DNS, and TLS.

Logging / secrets: an eck-operator-based EFK stack (Elasticsearch 3-node 3-AZ, EBS gp3, S3 snapshot SLM) runs HA, with External Secrets Operator externalizing secrets; every ServiceAccount authenticates via IRSA / Pod Identity, removing static keys.

Auth / deploy: ArgoCD integrates GitLab OAuth SSO + game-scoped RBAC; client artifacts ship via Jenkins → S3 + CloudFront, and server manifests reach the bastion's EFS mount via an S3 transit bucket + SSM SendCommand — the bastion is discovered dynamically by Name tag, so deploys use no SSH keys.

Troubleshooting

- Game-production Pods reserved ~4x the memory they actually used, causing node sprawl → right-sized memory requests from a measured profile, normalizing scale-out and improving spot cost efficiency
- During rolling deploys, ALB routed traffic to not-yet-ready Pods → combined ALB Pod readiness gate + PodDisruptionBudget + graceful drain to secure zero-downtime deploys

Results

- Expanded from a single on-premises dev cluster to an on-premises + AWS production multi-cluster hybrid, establishing the cloud production launch foundation for the new externally-published game
- Managed ~11 platform components 100% declaratively via GitOps (ArgoCD), and established static-key-free authentication with IRSA / Pod Identity + SSM SendCommand keyless deployment, removing bastion SSH-key leak risk
- ArgoCD app markers plus a production CI/CD transition doc make this a solo build the team can still operate and rebuild by the same procedure — a documented hand-off path

Project 9: Observability Hardening — Distributed Tracing & Progressive Delivery

Contribution 100% (Solo design & build — OTEL tracing pipeline · Argo Rollouts Canary · soft-launch operation) 2026.06 ~ Present

On top of the AWS EKS production game-launch platform (Project 8), strengthened observability and delivery for the new externally-published game's soft launch.

Filled the missing 'traces' axis of the logging/metrics-only observability platform with an end-to-end OpenTelemetry distributed-tracing pipeline, introduced zero-downtime Canary deployment via Argo Rollouts, and handled real soft-launch traffic on the stack without incident.

9-1. End-to-End Distributed Tracing Pipeline (OTel + Tempo)

Problem

The central observability platform had logging (EFK) and metrics (kube-prometheus-stack) but no 'traces' axis, so there was no way to follow how a request flows between services or where latency arises.

Ahead of the production launch, a means to diagnose inter-service latency and errors was needed.

Approach

Built a distributed tracing stack on AWS EKS: OpenTelemetry Collector + Grafana Tempo (S3 storage via EKS Pod Identity — no static keys) + OpenTelemetry Operator (auto-instrumentation injects language SDKs with no app image change). The key decision was where to sample: RED metrics are generated from 100% of spans before sampling for accuracy, while tail sampling (all errors + everything over 500ms + 10% of the rest) applies only at the trace-storage step to cut storage cost.

Wired Metrics ↔ Traces ↔ Logs into a 3-signal pivot in Grafana, filling the missing 'traces' axis of the prior logging/metrics platform (Project 3).

Results

- Added the 'traces' axis to a logging/metrics-only platform, wiring Metrics ↔ Traces ↔ Logs into a 3-signal pivot
 - Separated RED metric generation (100% of spans, pre-sampling) from trace storage (tail-sampled), keeping metric accuracy while cutting storage cost
 - Injected language SDKs with no app image change via auto-instrumentation, and accessed S3 with no static keys via EKS Pod Identity
-

9-2. Progressive Delivery with Argo Rollouts (Canary)

Problem

The production game service had to keep serving live traffic during deploys, and needed instant rollback when something went wrong. Plain rolling updates made it hard to control the transition so old and new safely coexist while traffic moves over.

Approach

The prior Blue-Green swapped the ALB target group all at once on promotion, so healthy targets momentarily dropped to zero and the ALB raced into 503s (observed 2026-07-14). That is why it was switched to Argo Rollouts Canary.

On the same ALB target group, `maxUnavailable: 0` + add-before-remove (register the new Pod before removing the old) lets old and new briefly coexist during the switch, removing the 503 race; the game service is stateless HTTP (state externalized via JWT + Redis/TypeORM), so it stays safe even while traffic is split across old and new. Templated the deployment strategy as an opt-in toggle in the shared base-aws Helm chart and enabled it on just the one production game main service (a sibling internal service with no public traffic keeps a plain Deployment), with the controller running leader-election HA + PDB so rollouts never stall through node drain / Karpenter consolidation.

Results

- Switched the production game service from Blue-Green to Canary, removing the 503 race where healthy ALB targets momentarily dropped to zero at promotion, and secured a zero-downtime switch via maxUnavailable:0 + add-before-remove on the same target group
 - Configured the controller as leader-election HA + PDB so rollouts never stall during node drain / Karpenter consolidation
-

9-3. Soft-Launch Operation (2026.07 ~ Present)

Results

- Soft launch went live 2026-07-14 (KST) — opened the new externally-published game to a limited release and handled its small-scale real traffic on the stack above with zero-downtime Canary deploys and 3-signal observability (verified trace_id/span_id correlation in live logs)
 - HPA · Karpenter autoscaling provisioned in preparation for the 2026 Q4 global launch (no scale-up triggered under soft-launch traffic)
 - Completed the observability, deployment, and autoscaling foundation for the 2026 Q4 (Sept-Oct) global launch
-

migration within a scheduled game-maintenance window with minimal downtime. Maintained dual source management — GitLab for on-premises, GitHub for production.

Project 1: Large-scale AWS → GCP Cloud Migration & CI/CD Build-out

Contribution: Infra architecture design, migration strategy & execution lead 70% (remaining 30% = server-team application-side migration collaboration), CI/CD pipeline construction 100% 10 months (2023.03 ~ 2023.12)

Problem

AWS costs were continuously rising (\$10,000+/month), particularly NAT Gateway and data transfer costs.

A GCP Startup Program opportunity then prompted the migration — targeting cost reduction alongside the global load balancing needed for the multi-region game service.

Approach

Established a 3-phase migration strategy executed sequentially — Phase 1: build and validate the dev environment on GCP, Phase 2: migrate QA, Phase 3: migrate production.

Designed the network on Shared VPC (Host Project / Service Project separation) and switched GitHub Actions' GCP auth to Workload Identity Federation, replacing service-account key issuance/rotation with short-lived OIDC tokens. Migrated DNS via subdomain delegation (Hosting.kr → AWS Route53 → GCP Cloud DNS) for pre-validation, then performed the final NS swap during a single scheduled maintenance window.

On cost and security, worked around Filestore's 2.5TB SSD minimum by self-hosting an NFS server on Compute Engine (pd-balanced 1TB) and configured Cloud Armor with region blocking + IP allow-list WAF policy.

Codified GCP infrastructure with Terraform (100% excluding IAM), then built the GitOps CI/CD pipeline combining GitLab CI and ArgoCD, standardizing environment configs with Helm Charts and establishing Git commit-based deployment history management.

Troubleshooting

- While migrating AWS ALB-based routing to a GCP Global LB, traffic-routing issues arose because AWS ALB still passes traffic to backends failing health checks whereas GCP LB blocks them.
Analyzed the GCP NEG (Network Endpoint Group) structure and reconfigured health checks/backends for GKE workloads
- GKE Ingress + BackendConfig health check returned 503 when the configured requestPath had no response.
Resolved by guaranteeing the health-check endpoint response and aligning ManagedCertificate / FrontendConfig to restore external ingress traffic
- Game client file downloads failed after the Cloud CDN migration due to a missing HTTP → HTTPS redirect.
Resolved by splitting the URL Map into HTTP / HTTPS variants (`default_url_redirect.https_redirect=true`) and mapping the HTTP forwarding rule to a dedicated port-80 target HTTP proxy
- After delegating DNS to GCP Cloud DNS, the domain was not automatically registered in Google Search Console, breaking search visibility and domain-ownership verification.
Resolved by adding the Search Console verification TXT record to Cloud DNS to verify ownership, then re-registering the domain

Results

- 50% cloud cost reduction (\$10,000 → \$5,000/month), ~\$60,000 annual savings
- Completed full service migration via the 3-phase strategy.
Minimized downtime with resource-copy based migration, and executed the final DNS cutover during an approximately 2-hour scheduled maintenance window aligned with game service operations
- Established IaC management for entire GCP infrastructure with Terraform (ensured code quality with TFLint/Checkov static analysis and Terratest unit testing)
- 50% deployment time reduction (40min → 20min), 95% rollback time decrease (30min → 1-2min)
- 3x weekly deployments, 100% deployment history tracking via Git commits
- Eliminated GCP service-account key management overhead and key-leak risk by adopting Workload Identity Federation for GitHub Actions, transitioning to a short-lived token security model

Project 2: Game Data Collection Automation System

Contribution 100% (Python script development & operations) 3 months (2024.01 ~ 2024.03)

Problem

Game and blockchain data was collected manually, with analysts spending 2-3 hours daily on data collection and frequent data gaps due to human errors.

Approach

Built a GCP-based pipeline that loads multi-source data (MongoDB · CloudSQL · Google Analytics · Dune) into BigQuery and automatically refreshes analyst-facing Google Sheets. MongoDB was ingested via a Dataflow Flex Template ETL, CloudSQL via BigQuery's direct connection feature, and GA / Dune via their respective APIs (Dune with a separate API key), normalized into a consistent index schema.

Python Cloud Functions push BigQuery query results into Google Sheets via the Sheets API, and Cloud Scheduler triggers them on Daily (previous-day data) / Monthly (previous-month data) cadences. A separate dedup Cloud Function runs periodically to keep BigQuery data

consistent. The full infrastructure (BigQuery datasets, Cloud Functions, schedulers, Cloud Storage, Artifact Registry, IAM bindings, service accounts) is managed as Terraform IaC for reproducible environments and change tracking.

Troubleshooting

- The CloudSQL → BigQuery direct connection (federated query) failed on region mismatch and missing connection-SA permissions. Resolved by creating the connection in the same region and granting the connection SA the Cloud SQL Client role
- Dune API's async execute → poll → fetch flow plus rate limits caused missing results / timeouts. Guaranteed result retrieval via execution-status polling with backoff and moved the API key into Secret Manager
- Daily re-runs re-loaded duplicate rows into BigQuery (non-idempotent). Applied MERGE / ROW_NUMBER() dedup logic in the Cloud Function to make loads idempotent, keeping zero gaps/duplicates
- Loading BigQuery results into Google Sheets failed on the 10M-cell-per-sheet limit, the write quota, and the per-minute request quota (429). Worked around the cell limit with chunked values.batchUpdate writes and sheet splitting, and stayed within quota via call pacing/throttling, batching, and exponential backoff

Results

- 95% data collection automation (eliminated 2-3h daily manual work)
- 0% data gap rate through automation eliminating human errors
- Terraform IaC for the full infra enabled rebuilding the same pipeline in another GCP project within 30 minutes, with 100% change history in Git

Project 3: PLG Stack Monitoring System

Contribution 100% (System design & implementation) 4 months (2024.04 ~ 2024.07)

Problem

No real-time monitoring system for Kubernetes clusters and applications, allowing only reactive incident response with excessive time spent on root cause analysis.

Approach

Built a PLG Stack collecting metrics with Prometheus and logs with Loki, unified visualization through Grafana dashboards. Real-time monitoring of CPU/memory/network usage and application logs, with automatic Slack alerts on threshold breaches.

Results

- Established proactive incident detection through threshold-based alerts
- Reduced incident response time through real-time monitoring and alerting
- Improved service availability
- Reduced operations team fatigue and improved actual incident detection accuracy through SLI/SLO establishment and Alertmanager alarm noise optimization

Iorchard

2022.04 ~ 2023.03

Infra Engineer

Built PaaS (Kubernetes + OpenStack + Ceph) infrastructure solutions for clients and provided technical support. Designed and built on-premises cloud infrastructure for financial and public sector clients, and led training for client operations teams.

Project 1: Customized PaaS Solution for Clients

Contribution 100% (Infra design & Kubernetes cluster build) 11 months (2022.04 ~ 2023.03)

Problem

Each client had different environment and server spec requirements, making standardized solutions insufficient.

Approach

Designed Kubernetes clusters in HA (High Availability) configuration per client requirements and built virtual machine management environments with OpenStack. Provided unified block/file/object storage through Ceph and conducted parallel infrastructure operations training for client teams.

Results

- Successfully built and delivered PaaS solutions to 5 clients
- Achieved stable operation with Kubernetes cluster HA configuration
- Ansible-based server provisioning and configuration management automation reducing per-client deployment time and achieving zero configuration drift

Project 2: Kubernetes Operations Training Program for DP

Contribution 100% (Training design & delivery) 6 months (2022.06 ~ 2022.11)

Problem

DP's operations team lacked Kubernetes and container-based infrastructure experience, anticipating difficulties in self-operation after PaaS solution adoption.

Approach

Designed a step-by-step training curriculum from Kubernetes basics to cluster operations and troubleshooting, with hands-on lab environments for practice-oriented training.

Also conducted parallel training on OpenStack and Ceph storage operations to ensure full PaaS stack operational capability.

Results

- Operated the curriculum with an average of 10 attendees per session and 4 hours per session, establishing DP's autonomous Kubernetes operations system
- Reduced client technical inquiries post-training, achieved self-incident response capability
- Standardized training materials reused for subsequent client trainings

Project 3: Kubernetes Certificate Auto-renewal Process Improvement

Contribution 100% (Solo execution, propagated across all client clusters) 2 weeks (2022.11)

Problem

Kubernetes cluster certificates expired annually, requiring ~5 hours of renewal work per client per year, with service outage risks upon expiration.

Approach

Analyzed the Kubernetes certificate management process and modified kubeadm settings to extend certificate validity from 1 year to 10 years.

Developed automation scripts applicable to existing clusters and deployed to all clients.

Troubleshooting

- Editing kubeadm settings only worked on kubeadm-initialized clusters, and on v1.15.x / v1.16.x a known `kubeadm alpha certs renew` bug left some certificates un-renewed. On those versions, rather than relying on the kubeadm command, I adopted a public open-source script (`update-kube-cert`) that backs up `/etc/kubernetes` and regenerates the certificate files directly to 10-year validity. Applied it together with restarts of etcd, apiserver, controller-manager, scheduler, and kubelet so it works regardless of version.

Results

- 10x certificate renewal cycle extension (annual → once per 10 years)
- ~50 hours annual operations savings across 10 clients
- Eliminated outage risks from certificate expiration